Emerging Technologies and Frameworks (ETF)

Emerging technologies

- This is the technology that is being currently developed or will be developed within the next five to ten years, and will alter the way the world works

3 “Key” laws

- **Moore’s law**
  - This was originally made in the form of an observation and forecast
  - The main concepts is that:
    - The density of transistors double every two years
    - The minimum cost for the complexity of the transistors doubles every year
- **Metcalfe’s law**
  - The higher the number of nodes (users) on the network, the value/power is square the number of nodes (users)
- **Gilder’s law**
  - He states that the bandwidths grows three times faster than the computer power

Cloud computing

- This is a way of providing on-demand computing services over the internet with a pay-as-you-go system.
- Users can use any services without having to own the equipment to use it, as that is done by third party companies.

Types of cloud computing

There are three types of cloud computing:

- Infrastructure as a service (IaaS)
    - This is contains the basic building blocks of cloud IT
    - Provides network features, computer, and data storage space
    - Has the highest level of flexibility
    - This is the most familiar type of IT resources which developers use in current time
- Platform as a Service (PaaS)
    - The user does not need to manage the infrastructure of the service, the resources, capacity, and software maintenance will all be done by the provider
    - The user only needs to focus on the deployment and management of the application
- Software as a Service (SaaS)
    - This provides a complete product that is run and managed by the service provider
    - SaaS is in most cases referred to as the end-user application
    - Only the use of the software has to be thought of by the user, everything else is handled by the service provider

Public, Private, and Hybrid cloud

- Public clouds is when all the services provided are provided over the internet
- Private cloud is when when the user deploys there own infrastructure and applications into their own data servers

- This is a mix of both **public** and **private** cloud

Virtualization

- This is the technology that allows for services to be created as virtual versions of themselves so it can be shared over the internet
- All these separate sections run on one single physical machine but act as independent systems
- There are three main types of virtualization in cloud computing:
    - Network virtualization:
        - This makes virtual networks that do not need physical hardware
    - Server virtualization:
        - This divided a physical server into multiple virtual servers
    - Storage virtualization:
        - This combines multiple storage devices into one virtual storage unit

Cloud attributes

There are five key attributes to cloud computing:

1. Shared resources
2. On-demand access
3. Metered by use
4. Scalable
5. Large network access

Quantum computing

A quantum computer is a machine that runs calculations based on the laws of quantum mechanics

Quantum binary systems

- Bits in quantum computing are called **Qubit (Quantum bit)** and are carried as atoms, ions, photons, or electrons and work together as the computer memory and processor

Quantum principles

- **Quantum uncertainty** → this is when the positions and velocity of a particle is unknown until observed
- **SUPERPOSITION** → this is when an atom can be both in a state of one (1) or zero (0) at the same time as there is an equal opportunity until it is observed
- **Quantum nonlocality-Entanglement** → this is when there are two or more particles that share similar properties and even if splits, will have those same properties

Comparing quantum and classical computing

- **Classical computer** → this is a devices that sends electricity through circuits and gates which can be calculated with classic mechanics
- **Quantum computer** → this is a device that uses quantum mechanics to perform operations on data using superposition and entanglement

3D printing

Industry revolution

- **First industrial revolution**
    - This began in the late 18th century, when the textile industry got mechanization
- **Second industrial revolution**
    - This came in the early 20th century with Henry Ford, who created mass production with the help of the moving assembly line
- **Third industrial revolution**
    - The is when manufacturing started to go digital and happened in the 70s

- **Industry 4.0**
    - This is when technology started to grow
    - Clever software, robots, novel materials, and three dimensional printing all started to be created

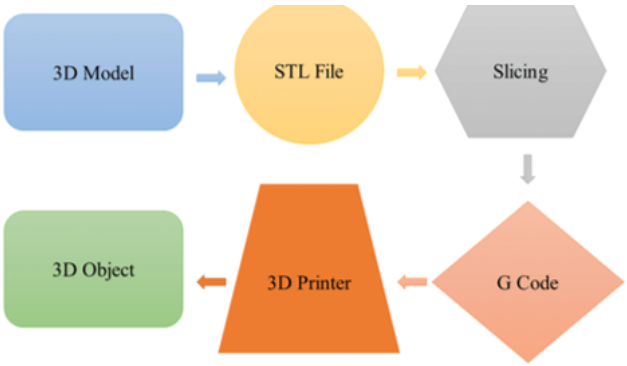
3D printing

3D printing is when items a manufactured using a design and materials are joined layer-by-layer to create the object

Manufacturing types

- There are two types of 3D manufacturing:
    - **Additive** → this is when layers are added to create the item
    - **Subtractive** → this is when material is removed using chemicals or physically

Steps in 3D printing



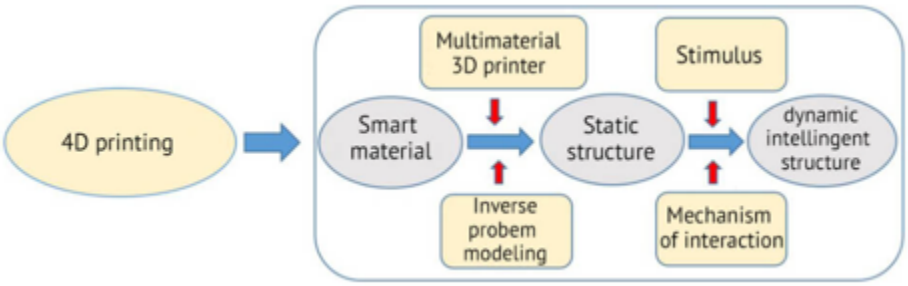
Applications of 3D printing

- Food processing
- Fashion
- Medical
- Art
- Electronics
- Industrial goods

4D printing

This is when a 3D printed object has the ability to transform into another structure with the introduction of an external energy source like temperature, light, or an environmental effect

How 4D printing works



Advantages of 4D printing

- The size of the object can change
- New material cans be used, which makes new properties

Possible applications of 4D printing

- Self repairing pipe systems
- Medical industry
- Fashion
- Self-assembly furniture

Modern AI

The turing test

- This was a test where a person chats with two other unseen players but one of the unseen players is a computer. The main player has to guess which player is a computer. For the computer to not get caught it has to have a vast understanding of topics and how the human language works.
- The first ever computer to pass this test came in 1966 and was known as ELIZA

What is AI?

- AI is the way of making machines think rationally, think like people, act like people, and act rationally
- Humans make rational decision, AI is a way of trying to create that rational decision making in machines
- Rational decision making occurs when a predefined goal can be achieved to its maximum potential with what is possible.

AI fields

- There are two types of fields in AI:
    - Natural language processing
        - Automatic speech recognition (ASR)
        - Text-to-speech synthesis (TTS)
    - Language processing technologies
        - Web search
        - Machine translation
        - Question and answering systems

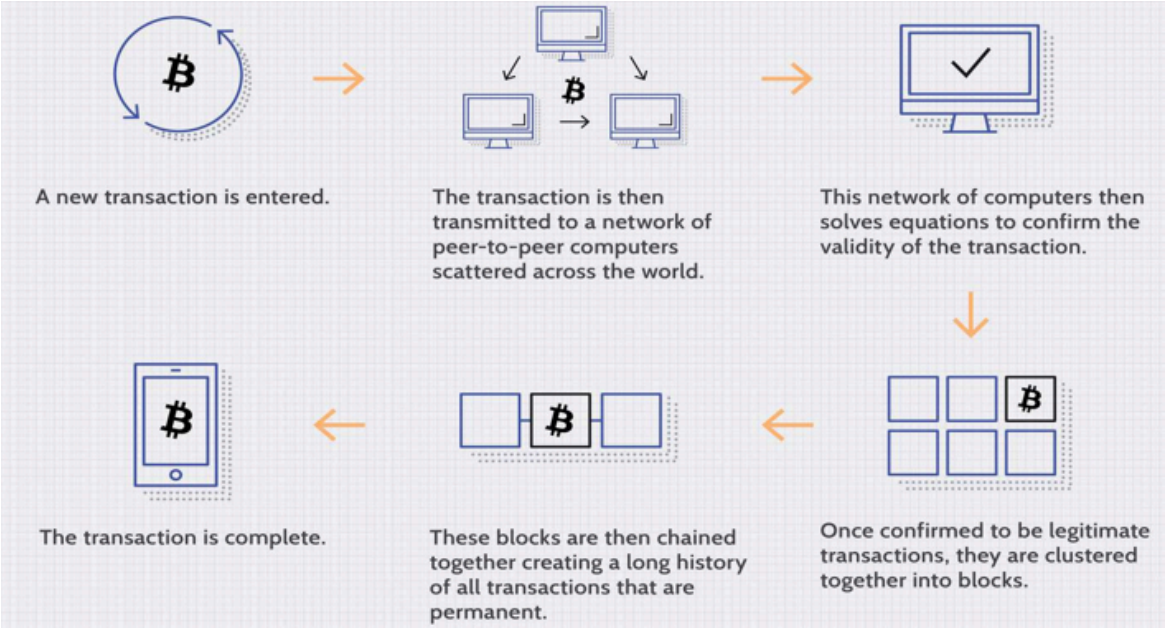
Blockchain & metaverse

Blockchain

- Blockchains are databases or ledges that are shared on a computer network node
- They are mainly known for be implemented in cryptocurrency systems to maintain secure and decentralized records of transaction
- They can be found in other sectors and not only cryptocurrency
- Blockchain can make the data of any company unable to be altered by outside sources

How blockchain works

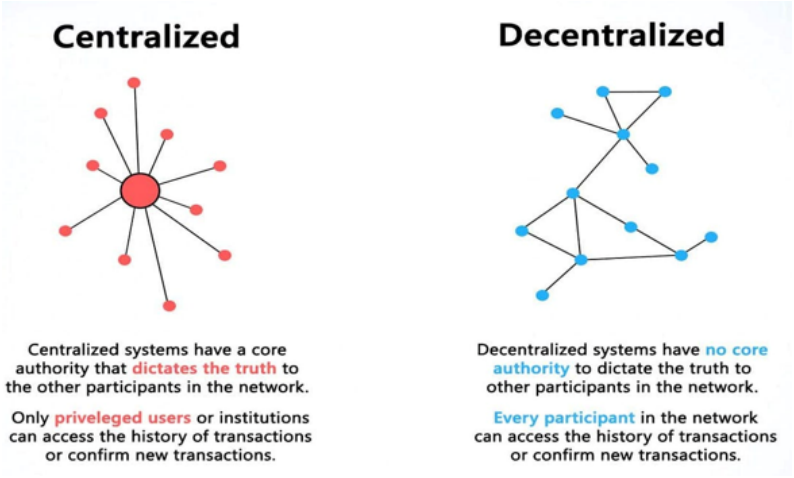
1. Blockchain structure:
    - Blockchains are decentralized databases where data is stored on multiple machines, which allow for more transparency and security.
2. Transaction process:
    - There are defined process a blockchain follows in transactions including initiation, storage in a memory pool, block closure, encryption, and validation through mining or other consensus mechanisms
3. Proof-of-work and conformation:
    - Proof-of-work is there to validate blocks by completing cryptographic puzzles, to make sure the blockchain is not altered



Blockchain decentralization

- Blockchain database are separated across many network nodes, which keep the data redundant and data fidelity
- Records cannot be altered in one database alone as the other nodes prevent that from happening, maintaining the integrity of the information

- No single node can alter any stored information alone, which increases the security and trust



Blockchain vs bitcoin

Bitcoin	Blockchain
Bitcoin is a digital currency that allows you to perform online transactions anonymously.	Blockchain is a digital public ledger that holds and catalogues all the Bitcoin transactions
It is a cryptocurrency that is not governed by a central authority or a central bank.	It is a technology that uses a distributed network of computers to hold transaction records.
It is a decentralized currency that works as a medium of exchange to secure transactions.	Although a Blockchain is inherently distributed, it is not necessarily decentralized.
Bitcoins cannot work without Blockchains.	Blockchain is a database for recoding transactions which is not limited to Bitcoins.

Blockchain applications

1. Financial services:
    - real-time transactions are done, simpler operations, and smart contracts are used which makes it more efficient and allows automation of financial processes

2. Retail and luxury resale:
    - Blockchians allow retailers to use NFTs to engage with customers and certify the authenticity of the luxury resale products, creating trust and transparency
3. Marketing, advertising, and healthcare:
    - Blockchains create more security and transparency in marketing, reducing the risk of frauds, and also improve healthcare processors like payment processors or data security, which optimizes patient care and efficiency

- Examples for blockchain applications are:
    - Bitcoin
    - Voting
    - Real estate
    - Online music
    - Law enforcement
    - Payment and transfers

Limitations of blockchains

1. Scalability:
    - In public blockchains there are limited transaction processors, different speeds, and affordability are all problems for blockchain adaptation
2. Energy consumption concerns:
    - More green and environmental machines are required for energy-incentive blockchain operations, as those operation raise environmental alarms
3. Interoperability hurdles:
    - Large blockchain platform need the best network interactions, which limits the technology's potential and data integration

4. Regulatory uncertainties:
    - Blockchain adoption is met with problems of regulations and anonymities, which is where a more balanced approach is required for innovation and sustainable development

Metaverse

- This is a 3D place that users can connect, engage, and have real world experiences through the technology of VR, AR, NFTs, and cryptocurrency
- Users use digital avatars with the help of VR/AR that give them a persense, allowing them to interact within a virtual environment
- The metaverse is a place where people can socialize, game, and work which can all have effects on both the real world and internet depending on the interactions of the user.

Features and functions

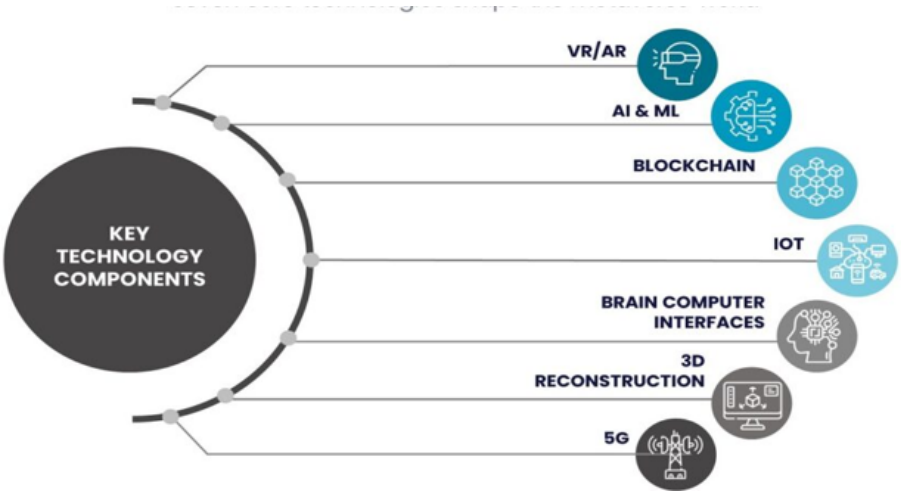
- The main goal of the metaverse is to create a place that people can socialize, create permanent virtual worlds, use blockchain technologies, and change reality
- Users can do real-life activities on the metaverse such as owing digital asset and properties, with the ease of being able to go anywhere in virtual locations

Technologies in metaverse

1. High-speed networks:
    - 5G & 6G networks are required to make the metaverse run smoothly and transmit high-quality streaming data and efficient content
2. Access devices:
    - Certain devices can only access the metaverse like augmented reality, virtual reality, and mixed reality devices. These offer the best interactivity and engagement

3. Diverse protocols:
    - Any application of content delivering system that is built for the metaverse must have a vast amount of protocols and languages, that all them to be compatible and intractable across platforms

4. Blockchain-based marketplaces:
    - The metaverse will have blockchain market places built into the systems that use cryptocurrency, which would allows users to buy and sell virtual goods and properties with NFTs, create a decentralized economy within the virtual world



The future of metaverse

1. Investments in future technologies:
    - Higher profits are generated by continuous investments, which driving innovations and development in metaverse related sectors

2. Increase user adoption:
    - There is predicted to be a large increase in users that use the metaverse technology by 2040, surpassing the current users levels and indicating that the technology is more accepted

3. Pandemic-driven technology acceleration:
    - More technology is created and accepted in times of pandemics mainly in healthcare, business, and education, which can grow the potential of metaverse applications in many places
4. Unlimited potential of AR and VR:
    - Many opportunities of creativity and innovations are given bt augmented reality (AR) and virtual reality (VR), which allows for more applications to be created across many industries, growing the metaverse

IoT (Internet of Things)

Smart cities

- This is an urban area where the use of technology and data collection helps improve the quality of life

Industrial revolution

- **Industry 1.0**
    - This is when machine were invented which replaced human labour
    - It happened in the 1700s and began in england, was when many good were created by machines
- **Industry 2.0**
    - This was the time more technological advancements started
    - Things like oil, steel, combustion engines, electricity, and rapid communications were created
- **Industry 3.0**
    - This was known as the digital revolutions for many systems were created and memory-programmable computers and controls were used in automation

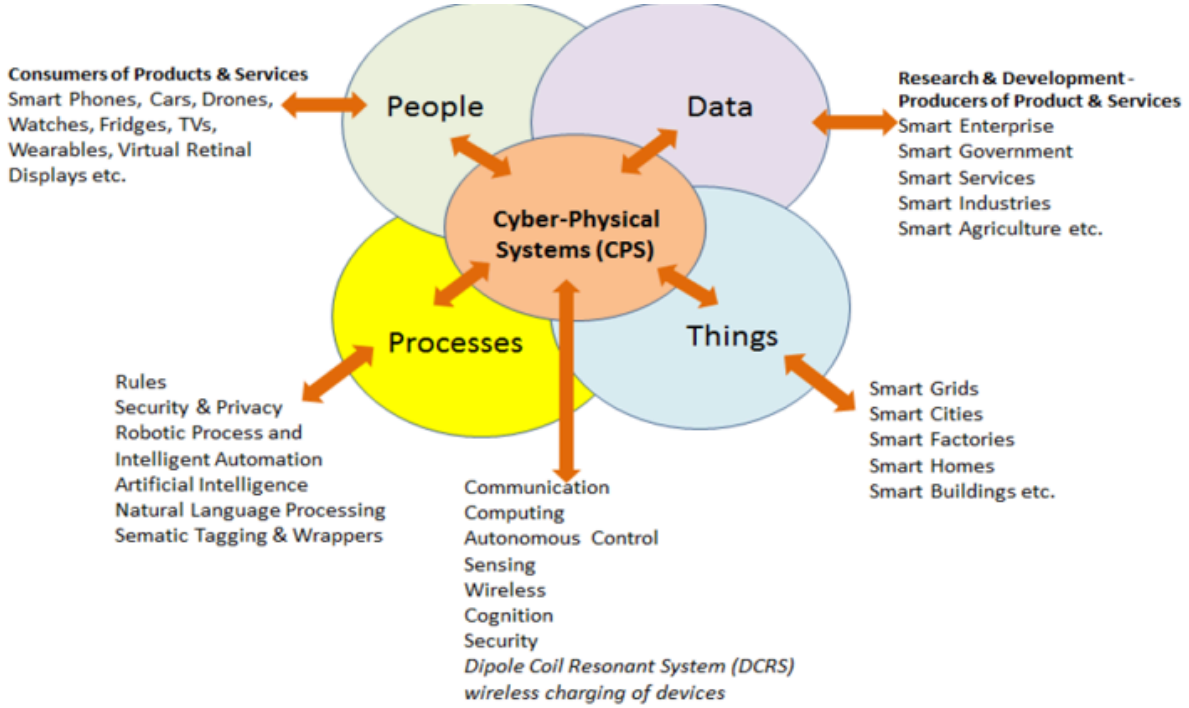
- Inventions like the computer, microprocessors, digital phones, and the internet were created in this time

- **Industry 4.0**
  - This is known as the automation revolution as it is the time were programs had started to be run by computer algorithms
  - Things like robots, VR, and the IoT (Internet of Things) were created in this time

- The **IoT** has the base from the system in industry 3.0 but move into industry 4.0

Cyber Physical Systems (CPS)

- These are systems that creates interconnected things with many different technologies together
- Both the physical and virtual worlds are put together
- CPS requires a device, network, and user to all work together on one single device



Converging technologies

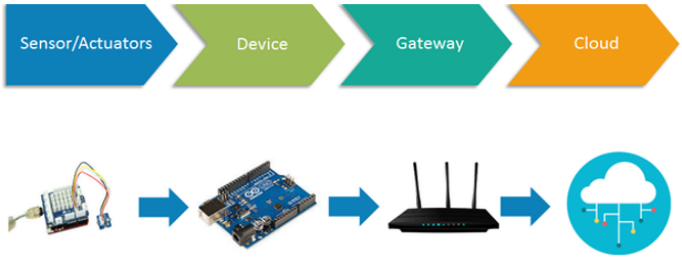
- **Big data** → this is the data that is to large, fast, or complex that it makes it to difficult or impossible to process using traditional methods
- **Cheap hardware** → this refers to low costing computer devices which are affordable and usable by many people. Cheaper hardware allows for more people to use technology and more innovation can be achieved
- **Processing power** → this refers to the capabilities of a system or device, normally measured through how many instructions it can perform in a set time limit. Advancements have made it possible for technologies to now run multiple complex or resource-intensive applications at set times.
- **Bandwidth** → this refers to the maximum data transfer rate of the network connection (BPS)/ the higher the bandwidth the faster the data transmission and network performance. This allows for real-time communications. Streaming media, and handling large data transfers

What is IoT (Internet of Things)

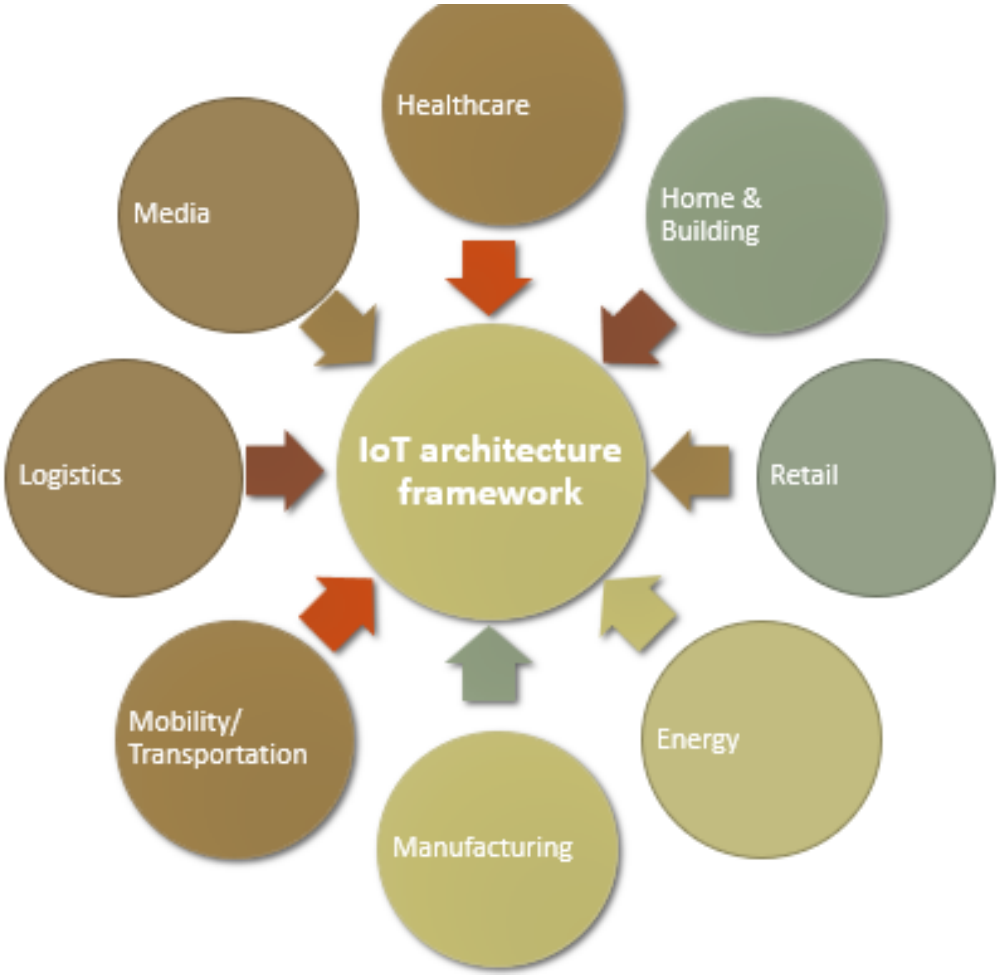
- This is an interconnect network of physical objects or embedded electronics, software, sensors, and network connectivity, which works together to allows these objects to collect and exchange data
- As long a physical object is connected to the internet to be controlled or communicate information, it can be considered as an IoT device

Physical object + Controller sensor and Actuators + Internet = Internet of Things

IoT stages



IoT application domains



Sensor in operation

- A **sensor** is a device that detects changes in the environment
- The **sensor** measures the phenomenon and converts that into electric signals.
- Sends raw data
- Low power

Sensor to actuator

- This is a device that converts energy into motion
- It is usually used to apply force on something

Device for local processing

- Get data from sensors
- Process
- Send data to cloud
- Small programmable devices

Network and internet

- IoT gateway
- Gather data from sensors
- Protocols:
    - CoAP
    - MQTT
    - HTTP

M2M vs IoT vs IoE

Feature	M2M (Machine-to-Machine)	IoT (Internet of Things)	IoE (Internet of Everything)
Definition	Direct communication between machines/devices without human intervention.	A network of connected devices that collect, analyze, and share data over the internet.	A broader concept integrating people, processes, data, and things for enhanced decision-making and automation.
Scope	Device-to-device communication.	Device-to-cloud or device-to-device communication using internet protocols.	Holistic system integrating IoT with human and business intelligence.
Connectivity	Uses wired or wireless connections (e.g., RFID, Bluetooth, cellular).	Uses the internet, cloud computing, and wireless technologies (Wi-Fi, LPWAN, etc.).	Uses a combination of IoT, AI, Big Data, and advanced networking technologies.
Communication	Point-to-point communication.	Networked communication with cloud-based data processing.	Intelligent and dynamic communication across multiple entities (people, data, devices, processes).
Human Involvement	Minimal to none.	Involvement in data interpretation and decision-making	High integration of human intelligence and decision-making automation.
Example Use Cases	Smart meters, industrial automation, ATMs, remote monitoring.	Smart homes, wearables, connected vehicles, healthcare monitoring.	Smart cities, intelligent transportation, smart healthcare, digital transformation in businesses.

Responsive web design

- This is when the web page automatically adjusts itself so that it looks good on any device

- Bootstrap is the most popular HTML, CSS, and javascript framework for creating responsive web sites

What is bootstrap

- This is a free front-end framework for faster and easier web development
- It includes both HTML and CSS design templates as well as optional Javascript plugins
- Responsive designs can easily be created with bootstrap

Bootstrap history

- It was created by Mark Otto and Jacob thornton
- The main advantages are:
    - Easy to use
    - Responsive features
    - Mobile-first approach
    - Browser compatibility

Getting bootstrap

- There are two ways to get bootstrap:
    - Download bootstrap from getbootstrap.com
    - Include Bootstrap from a **CDN**

jQuery

- This is the most famous Javascript framework, which was made to change the way Javascript is written
- **jQuery** is a Javascript library
- **jQuery** simplifies the amount of code needed to write Javascript with
- It is also easier to learn

Basic syntax

- **$ (selector).action ( )**

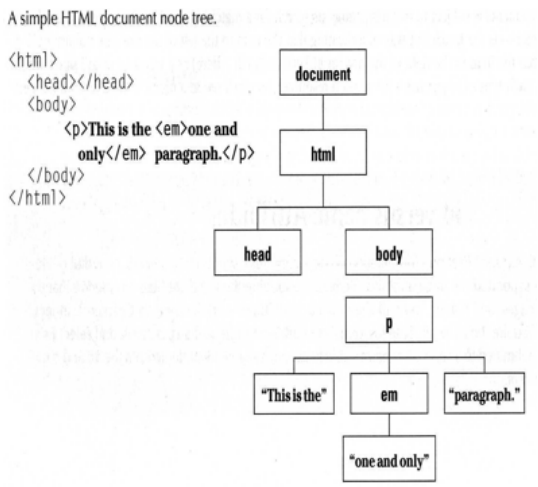
- The **$** is used to define/access jQuery
- A (selector) is used for “query command” in HTML elements
- **jQuery action ()** is used on elements

How does it work

- jQuery relies on the structure of the HTML DOM

DOM

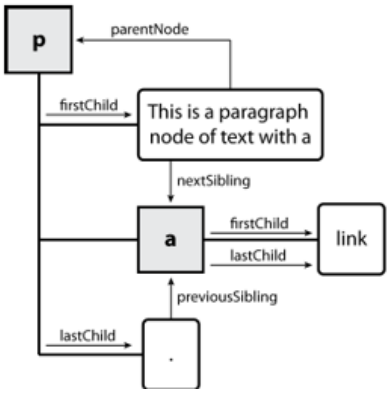
- This is a structure that shows the object in the code. The elements of the DOM can be manipulated within the syntax if the programming language used.



Traversing the DOM tree

- There is a certain hierarchy in a dom tree

Name(s)	Description
firstChild, lastChild	start/end of this node's list of children
childNodes	array of all this node's children
nextSibling, previousSibling	neighboring nodes with the same parent
parentNode	the element that contains this node



Document ready handler

- A **ready** event always occurs when the DOM has been loaded
- This is a good place to have all the jQuery event and functions as this event occurs after the document is ready
- The **ready ( )** method specifies what happens when a **ready** event occurs

```
<script>
    $(function() {

    });
</script>
```

Short Version

```
<script>
    $(document).ready(function(){

    });
</script>
```

Long Version

jQuery events

Mouse Events	Keyboard Events	Form Events	Document/Window Events
click	keypress	submit	load
dblclick	keydown	change	resize
mouseenter	keyup	focus	scroll
mouseleave		blur	unload

JSON

- **JSON** is used more now as it is lightweight, easy to understand, manipulate and generate.
- It replaces **XML** which was the previous format used as the data-interchange format
- **XML** is heavier than **JSON**
- **XML** requires xpath to operate, which is removed in **JSON** as it is native to javascript
- **XML** uses tags to describe user data which increase the size of the data

- An example of **JSON** string is:

  ‘{“name”:John”,”age”:30,”car”:null”}’

  - There are 3 properties defined in the object
- Each property has a value
- **JSON** string can be parsed with Javascript programs to access data as objects

JSON structures

- **JSON** is built in two ways:
  - **A collection of name/value pairs:**
    - This can be seen as an object, record, dictionary, or associative array
  - **An ordered list of values:**
    - This can be seen as an array, vectors, list, or sequence

Syntax of JSON

- **Object**
  - This is an unordered set of name/value pairs
  - An object begins { (left bracket) and ends with } (right bracket)
  - Each name is followed by a : (colon) and the name/value pairs are separated by , (comma)
  - Eg. **{name:”John”, age:31, city;”New York”};**
    - To access a JS object use:
      - **person.name**
- **Array**
  - This is an ordered collection of values
  - The array begins with [(left bracket) and ends with] (right bracket)
  - Values are separated by , (comma)
  - Eg.

```
{“employee”:[
        {“firstName”:”John”,”lastName”:”Doe”},
        {“firstName”:”Anna”,”lastName”:”Smith”},
        {“firstName”:”Peter”,”lastName”:”Jones”}
]}
```

- A value can be a string in double quotes, a number, or true or false or null, or an object or an array. The structure can also be nested.
- A string can be a collection of zero or more unicode characters, wrapped in double quotes.

XML examples

```
<?xml version="1.0" ?>
<students>
   <student>
         <id>1</id>
         <name>Adnan</name>
   </student>
   <student>
         <id>2</id>
         <name>Irfan</name>
   </student>
</ students>
```

XML to JSON

XML

```
<empinfo>
   <employees>
      <employee>
         <name>James Kirk</name>
         <age>40></age>
      </employee>
      <employee>
         <name>Jean-Luc Picard</name>
         <age>45</age>
      </employee>
      <employee>
         <name>Wesley Crusher</name>
         <age>27</age>
      </employee>
   </employees>
</empinfo>
```

JSON

```
{   "empinfo"  :
      {
            "employees" :   [
            {
                  "name" : "James Kirk",
                  "age" : 40,
            } ,
            {
                  "name" : "Jean-Luc Picard",
                  "age" : 45,
            } ,
            {
                  "name" : "Wesley Crusher",
                  "age" : 27,
            }
                              ]
      }
   }
```

Validating JSON & JSON security

- **Eval ( )** is an inbuilt Javascript method that can be used to validate JSON strings

Benefits of JSON over XML

- **JSON** supports data types like string, integer, Boolean etc.
- **JSON** is native to Javascript which makes it faster for the browser to read and understand
- There are no tags in **JSON** so only the data is transferred between the client and server, so it is lighter than **XML**
- Easy for humans to read and write

AJAX (Asynchronous Javascript And XML)

- This is not a programing language but mainly used as a development technique for creating interactive web applications
- Javascript is used to send and receive data between web browsers and web servers
- **AJAX** allows for web pages to be updated without reloading the whole page, as the data is being exchanged with the server behind the scenes. This makes the web page more responsive.

How to use Ajax?

1. **Creating an XMLHttpRequest object**
    - Variable = new XMLHttpRequest ( );
2. **Define a callback function**
    - xhttp.onload = function ( ) {
      // what to do
      }
3. **Open the XMLHttpRequest object**
    - xhttp.open("GET", "ajax_info.txt");
4. **Send a Request to a server**
    - xhttp.send ( );

XMLHttpRequest methods

open ("method", "URL", [async, username, password])

- Assigns destination URL, method, etc.

send (params)

- Sends request including potable string or DOM object data

abort ()

- Terminates current request

getAllResponseHeaders ()

- Returns headers (name/value pairs) as a string

getResponseHeader ("header")

- Returns value of a given header

setRequestHeader ("label","value")

- Sets Request Headers before sending

XMLHttpRequest properties

onload - Defines a function to be called when the request is received (loaded)

onreadystatechange - Defines a function to be called when the readyState property changes

readyState - Holds the status of the XMLHttpRequest.

0 = uninitialized 3 = interactive (some data has been returned)
1 = loading (broken in IE 6.0)
2 = loaded 4 = complete

status - HTTP Status returned from server:

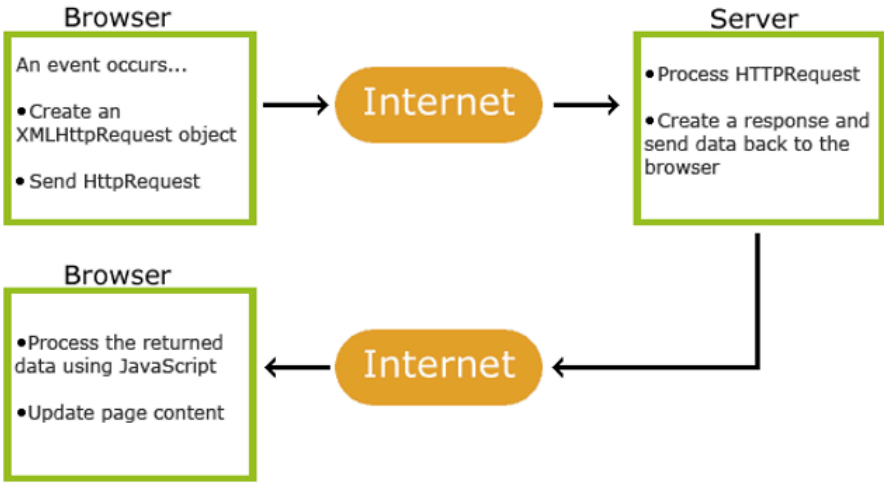
200: "OK"
403: "Forbidden"
404: "Not Found"
For a complete list go to the Http Messages Reference

responseText - Returns the response data as a string

responseXML - Returns the response data as XML data

statusText - Returns the status-text (e.g. "OK" or "Not Found")

How it works



jQuery - AJAX

- These methods allow request text, HTML, XML, or JSON from a remote server using both HTTP Get and HTTP Post

jQuery load ( ) method

- The method is called **load ( )**
- This method loads data from a server and puts the returned data into the selected element

[selector].load (URL, [data], [callback]);

Getting JSON data

- The function is called **getJSON ( )**
- This allows the returned JSON string and makes the resulting string available to the callback function as the first parameter to take further action

[selector].getJSON (URL, [data], [callback]);

jQuery ajax ( ) method

- This method sends asynchronous HTTP requests to the server

REST based web services

- **REST** means **RE**presentational **S**tate **T**ransfer
- Resources are used instead of web pages
- Each resource has a unique identifier (URI)
- It is an architectural pattern

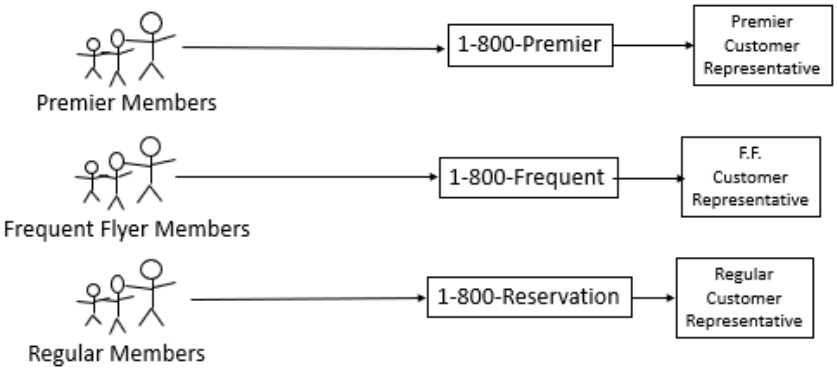
Example

Ticket reservation service

- An airline wants to create a telephone reservation system for customers to call and make flight reservations
- The airline wants to make sure premier members get immediate services, frequent members get expedited services, and others get regular services

Telephone Numbers are Cheap! Use Them!

The airline provides several telephone numbers - one number for premier members, a different number for frequent flyers, and still another for regular customers.

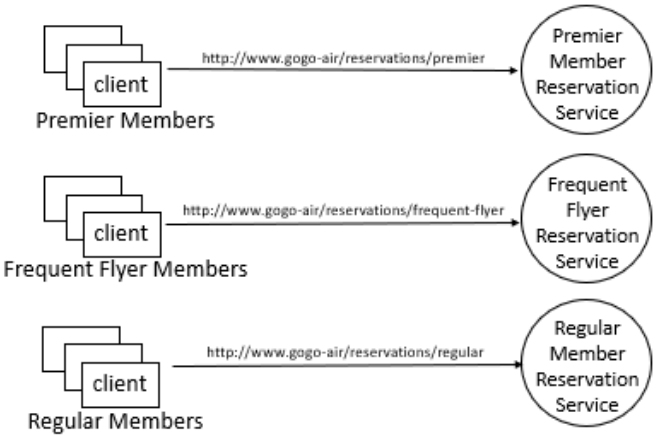


Web-based reservation service

- Now the airline wants to create a web reservation service for the customers to make flight reservations
- The same services are still in place for all the customers

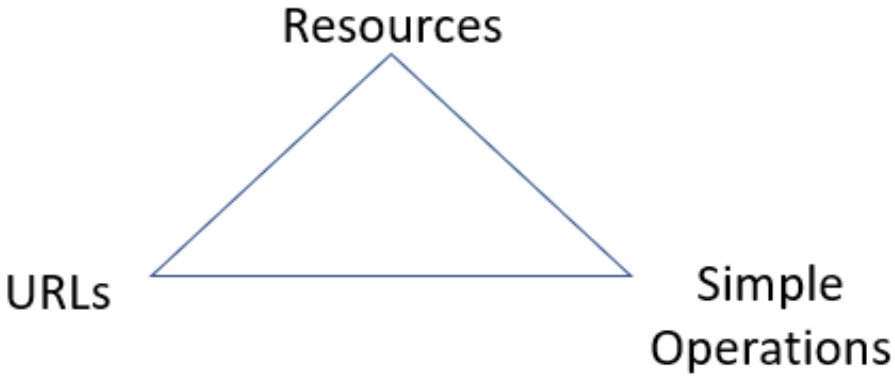
URLs are Cheap! Use Them!

The airline provides several URLs - one URL for premier members, a different URL for frequent flyers, and still another for regular customers.



Fundamental aspects of the REST design pattern

- **Resources**
  - Every entity is a resource, the resource may be a web site, a HTML page, a XML document, a physical device, etc
- **URLs identify resources**
  - There is a unique URL for every resource, this is known as **Axiom 0** created by Tim Berners-Lee web design



REST operations

HTTP Verb	CRUD	Entire Collection (e.g. /customers)	Specific Item (e.g. /customers/{id})
POST	Create	201 (Created), 'Location' header with link to /customers/{id} containing new ID.	404 (Not Found), 409 (Conflict) if resource already exists..
GET	Read	200 (OK), list of customers. Use pagination, sorting and filtering to navigate big lists.	200 (OK), single customer. 404 (Not Found), if ID not found or invalid.
PUT	Update/Replace	405 (Method Not Allowed), unless you want to update/replace every resource in the entire collection.	200 (OK) or 204 (No Content). 404 (Not Found), if ID not found or invalid.
PATCH	Update/Modify	405 (Method Not Allowed), unless you want to modify the collection itself.	200 (OK) or 204 (No Content). 404 (Not Found), if ID not found or invalid.
DELETE	Delete	405 (Method Not Allowed), unless you want to delete the whole collection—not often desirable.	200 (OK). 404 (Not Found), if ID not found or invalid.

- **REST request - GET**
    - This is used to retrieve resources representation/information only
- **REST request - POST**
    - This is used to create a new resource into the collection of resources
- **REST request - PUT**
    - This updates an existing resource (if the resource does not exist, the API may create a new resource or not)
- **REST request - DELETE**
    - This is used to delete resources (it is identified by the request-URI)
- **REST request - PATCH**
    - This is used to identify specific items in the resource
    - If the resource does not exist, an error is given

REST API implementation

Node.js

- This is a cross-platform, Javascript environment on the server

- An event-driver is used in Node.js, not a non-blocking I/O model
- Node.ls is like a web browser with a different set of C++ modules
- The modules can be used for interacting with local system resources such as processes, file systems, networking, etc.

Node REPL environment

- **REPL** means read, evaluate, print, loop
- Users can use Javascript commands, evaluate it, print the results, and loop to enter another command
- Sessions can be saved, and reloaded again
- It supports multiple commands, customized command prompts, and preconfigured context

MongoDB

- This is a database where data can be stored and managed
- The data is stored documents instead of tables and has the format of a JSON file
- It is a NoSQL database, which means no rows or columns are used

Motivations

- **Problems with SQL:**
    - Rigid schema
    - Not easily scalable
    - Requires join operator, which is hard to understand
- **Perks of MongoDB:**
    - It has an easy interface with common languages
    - DB tech can be run anywhere
    - Keeps features of relational DBMS and learns key-values from NoSQL systems

Data model

- The data is stored in documents (max 16 MB)

- All documents are in BSON (Binary JSON) format
- Each document is stored in a collection (folder)

BSON example

```
{
"_id" :        "101",
"city" :       "TNE",
"pop" :        2660,
"state" :      "MB",
"councilman" : {
                 name: "John"
                 address: "13 Cross Street"
               }
}
```

The _id field

- This is a field in each document that has special characteristics:
  - It is the primary key for the collection
  - It is unique, immutable, and cannot be an array data type
  - The default data type is ObjectId, sorting on an ObjectId value is basically like sorting based on time of creation

MongoDB vs. SQL

MONGODB	SQL
Document	Tuple
Collection	Table/View
PK: _id Field	PK: Any Attribute(s)
Uniformity not Required	Uniform Relation Schema
Index	Index
Embedded Structure	Joins
Shard	Partition

CRUD

- These are the main functions that are performed when working with databases, it stands for **C**reate, **R**ead, **U**pdate, **D**elete.

  - Create
    - db.collection.insert(<document>)
    - db.collection.save(<document>)
    - db.collection.update(<query>, <update>, { upsert: true })
  - Read
    - db.collection.find(<query>, <projection>)
    - db.collection.findOne(<query>, <projection>)
  - Update
    - db.collection.update(<query>, <update>, <options>)
  - Delete
    - db.collection.remove(<query>, <justOne>)

CRUD: using the shell

- To check which db is being used, the command **db.** Is used
- To show all databases, the command **show bds** is used
- To create/switch db's, the command **use <name>** is used
- To see what collections exist, the command **show collections** is used
- To insert documents into a collection/make a new collection, the command format required is:
  - **db.<collection>.insert (<document>)**

CRUD: inserting data

- To insert one document, the command required is:
  - **db.<collection>.insert({<field>:<value>})**
- To insert multiple documents at once, an array is used

CRUD: Querying

- This is used to find data within the collection, the required command is:
    - **db.<collection>.find ({<field>:<value>})**
- The first 20 results will be displayed
- AND:
    - This is used to find specific documents in a collection, the command is:
        - **db.<collection>.find ({<field1>:<value1>,**

          <field2>:<value2>})
- OR:
    - This is used to find data that may be in different documents or field, the command used is:
        - **db.<collection>.find ({ $or: [**

          <field>:<value1>

          <field>:<value2>]

          })
    - To find values that may be in the field, the command needed:
        - **db.<collection>.find({<field>: {$in [<value>, <value>]}})**
- including/excluding document fields:
    - This is used to select certain fields from document, the command is:
        - **db.<collection>.find ({<field1>:<value>},{<field2>:0})**
        - **db.<collection>.find({<field>:<value>}, {<field2>:1})**

CRUD: updating

- This updates the data specified in the the document

```
db.<collection>.update(
{<field1>:<value1>},        //all docs in which field = value
{$set: {<field2>:<value2>}},                //set field to value
{multi:true} )              //update multiple docs
```

CRUD: removal

- This removes the record from where the field equals the value given, the command is:
    - **db.<collection>.remove ({<field>:<value>})**
- To only remove the first element use:
    - **db.<collection>.remove ({<field>:<value>}, true)**